

Algoritmi e Strutture Dati – Parte A – 02/02/26

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Scrivere l'equazione di ricorrenza associata a questo algoritmo e calcolare la complessità computazionale che ne deriva.

```

int mystery(int[] A, int start, int end)
{
    if end - start + 1 < 10 then
        int sumSoFar = 0
        for i = start to end do
            sumSoFar += A[i]
        return sumSoFar
    else
        int sumSoFar = 0
        sumSoFar += mystery(A, start + 10, end)
        sumSoFar += mystery(A, start, end - 10)
        return sumSoFar
}

```

A2 – Albero sommato – Punti ≥ 10

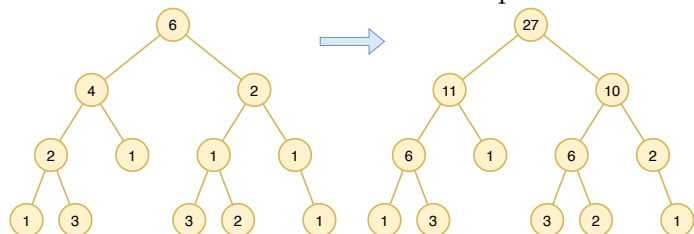
Sia dato un albero binario T in cui ogni nodo t contiene un valore intero $t.value$. Un *albero sommato* è un albero con la stessa struttura di T in cui il valore associato a ciascun nodo t è pari alla somma dei valori di tutti i nodi del sottoalbero radicato in t (incluso t stesso). Scrivere un algoritmo

TREE summedTree(TREE T)

che prenda in input l'albero T e restituisca un *nuovo* albero sommato T' .

Discutere informalmente la correttezza dell'algoritmo proposto e analizzarne la complessità computazionale.

La figura seguente mostra un esempio di albero di input e l'albero sommato da restituire in output.



A3 – Massimo due – Punti ≥ 12

Data una stringa S contenente n caratteri, si vuole determinare la lunghezza massima di una sottostringa contigua che contenga al più due caratteri distinti. Scrivere un algoritmo

int maxTwo(**item**[] S , **int** n)

che prenda in input una stringa S di dimensione n e restituisca la lunghezza della più lunga sottostringa contigua che contenga al più due caratteri distinti.

Discutere informalmente la correttezza dell'algoritmo proposto e analizzarne la complessità computazionale. Si noti che esistono soluzioni con costo $O(n^3)$, (n^2) , $(n \log n)$ (lunga da scrivere), $O(n)$. Alcuni algoritmi hanno complessità peggiori e migliori diverse, quindi prestate attenzione nell'analisi.

Nel caso di $S = \text{"abaccbb"}$, la sottostringa valida di lunghezza massima è "ccbb" , che ha lunghezza 5. Pertanto, l'output dell'algoritmo per questa stringa è 5.

Algoritmi e Strutture Dati – Parte B – 02/02/26

B1 – New York Dreaming – Punti ≥ 8

Vi state allenando per la maratona di New York e il vostro programma di allenamento prevede corse di lunghezza variabile.

Il sistema stradale è modellato come un grafo orientato $G = (V, E)$, in cui i nodi rappresentano punti di snodo e gli archi rappresentano strade percorribili. La lunghezza (positiva) di un arco (x, y) è data da una funzione di peso $w(x, y)$; se l'arco non esiste, $w(x, y) = \infty$.

Dato un valore k (con $10\,000 \leq k \leq 30\,000$), volete individuare tutti i cammini che:

- partono da casa vostra (nodo $s \in V$) e terminano nuovamente in s ;
- non ripetono nodi, con l'unica eccezione di s che compare esattamente due volte (inizio e fine);
- hanno lunghezza totale (cioè, somma delle lunghezze dei suoi archi) compresa nell'intervallo $[k - 500, k + 500]$.

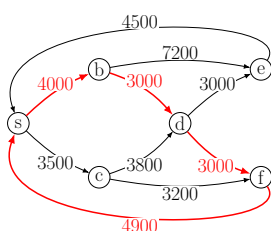
Scrivere un algoritmo

```
printPaths(GRAPH G, NODE s, int k)
```

che stampi tutti i cammini che seguono le regole descritte sopra.

Discutere informalmente la correttezza dell'algoritmo proposto e analizzarne la complessità computazionale.

Nell'esempio seguente, è evidenziato uno dei cammini possibili con $k = 15\,000$: $[s, b, d, f, s]$ ha infatti costo 14900. Oltre a questo cammino, l'algoritmo deve anche stampare $[s, b, d, e, s]$ e $[s, c, d, f, s]$.



B2 – Personale ospedaliero – Punti ≥ 10

Un ospedale deve assegnare il proprio personale di supporto ai reparti. Sono disponibili n operatori, divisi in due tipologie:

- n_I infermieri nell'insieme P_I ;
- n_O OSS (Operatori Socio-Sanitari) nell'insieme P_O .

Si noti che l'insieme degli operatori P è partizionato in infermieri e OSS, ovvero $P = P_I \cup P_O$ e $P_I \cap P_O = \emptyset$.

L'ospedale è organizzato in un insieme $R = \{r_1, \dots, r_m\}$ di reparti (ad esempio pronto soccorso, chirurgia, medicina, geriatria). Ogni operatore *deve essere assegnato a esattamente un reparto* e può essere assegnato solo a reparti per i quali è qualificato.

Ogni operatore p_i ha indicato un insieme

$$capable_i \subseteq R$$

di reparti in cui è in grado di lavorare.

Per ogni reparto r_j sono dati:

- un numero *minimo* di infermieri mi_j ;
- un numero *minimo* di OSS mo_j ;
- un numero *esatto* di operatori $m_j \geq mi_j + mo_j$.

Descrivere un algoritmo che verifichi se esiste un'assegnazione tale che:

- ogni operatore sia assegnato a esattamente un reparto per cui è capace;
- ogni reparto r_j riceva esattamente m_j operatori
- ogni reparto r_j riceva almeno mi_j infermieri e almeno mo_j OSS.

B3 – Hateville PNRR – Punti ≥ 12

Hateville è un villaggio lineare composto da n case. Il comune di Hateville ha ottenuto un finanziamento per ristrutturare le facciate di tutte le case.

La ditta incaricata dei lavori ha presentato un preventivo sotto forma di una matrice *cost* di dimensione $n \times k$, dove $k \geq 2$ è il numero di colori disponibili. Per ogni casa h e per ogni colore c , il valore $cost[h][c]$ rappresenta il costo per ristrutturare la facciata della casa h e dipingerla con il colore c .

A Hateville i vicini si odiano: per questo motivo, due case adiacenti devono essere dipinte con colori diversi. L'obiettivo del comune è minimizzare il costo totale della ristrutturazione, scegliendo un colore per ciascuna casa e rispettando il vincolo sui colori delle case adiacenti.

Scrivere un algoritmo

```
hateville(int[][] cost, int n, int k)
```

che, dato il preventivo *cost*, restituisca il costo minimo totale con cui è possibile ristrutturare tutte le n case.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.